CST8219 – C++ Programming

Lab2: Advanced Student Management System (Modern C++ & Menu Interface)

OBJECTIVE

This lab extends Lab 1: Student Management System by introducing modern C++ features, dynamic memory management with smart pointers, robust I/O validation, and a menu-driven interface.

By completing this lab, students will be able to:

1. Apply **unscoped (**enum**) and scoped enums (**enum class**)** in program logic.
2. Use auto, const, constexpr, static, and volatile appropriately.
3. Manage dynamic memory safely using **smart pointers**.
4. Traverse collections using **index-based loops, range-based loops, and iterators**.
5. Implement **robust input/output validation**.
6. Use **type casting** (static_cast) safely.
7. Format outputs using <iomanip> and produce a **class summary report**.
8. Implement a **menu-driven program** for continuous user interaction.

Pre-requisite:

- Completion of Lab 1 (Student Management System).
- Lab 2 builds on Lab 1; do not start a new project.

Data Model:

Each student record must include:

- **ID** (int)
- **Name** (std::string)
- **Marks** in 3 subjects (float[3])
- **Total** marks (float)
- **Average** marks (float)

- **Pass/Fail** status (Status enum)
- **Letter Grade** (Grade scoped enum)

Requirements:

1. Enums and Strong Typing

- Use **unscoped enum** for Pass/Fail:

```
enum Status { FAIL, PASS };
```

- Use **scoped enum** for Letter Grade:

```
enum class Grade { A, B, C, D, F };
```

- Provide a helper function to convert Grade to string for output.
- Do not implicitly convert scoped enums to integers.

2. Constants and Program State

- Use `constexpr` for number of subjects and passing threshold.
- Use a `static` variable to track **total students created**.
- Demonstrate `volatile` with a variable simulating external state (e.g., “file write in progress”).

3. Memory Management

- Student storage must use:

```
std::unique_ptr<Student[]>
```

- Internal arrays may also use

```
std::unique_ptr<float[]>.
```

- Raw `delete` must **not** be used in `main()`.

4. Traversal Techniques

- **Index-based loops**: input and file loading
- **Range-based loops**: display and formatting
- **Iterator-based loops**: searching and highest scorer
- Use `auto` where appropriate.

5. Input Validation

- Validate number of students (>0).
- Validate marks (0–100).
- Validate ID (positive integer).
- Validate menu choices and handle invalid inputs gracefully.
- Recover from invalid input using `cin.clear()` and `cin.ignore()`.

6. Menu-Driven Interface

Implement a **loop-based menu**:

```
===== Advanced Student Management System =====

1. Add student records

2. Display all students

3. Find highest scoring student

4. Display class summary

5. Save records to file

6. Load records from file

7. Exit

Enter your choice:
```

Menu Rules:

- Menu repeats until **Exit** is selected.
- Each option must call a **separate function**.
- Invalid menu options must display an error and re-prompt.
- A `switch` statement must be used for selection.

7. Output Formatting

- Display student records in **tabular format** using `<iomanip>`.
- Show average marks to **two decimal places**.
- Include **Grade** and **Status** columns.
- Include **class summary**: total students, class average, number passed.

8. Type Casting

- Use `static_cast` for numeric conversions, such as calculating averages and determining grades.
- C-style casts are **not allowed**.

9. File Handling

- Save student records to `students.txt` in structured format:

  ID | Name | Marks | Average | Grade | Status

- Loading must:
  - Validate data
  - Skip corrupt/incomplete records
  - Display error messages without crashing

Edge Cases to Handle:

1. Input & Validation Edge Cases

Scenario	Expected Behavior
Non-numeric menu input	Display error, clear input, re-show menu
Floating-point menu input	Reject and re-prompt
Menu option outside valid range	Display error and re-prompt
Entering 0 students	Reject input
Entering negative number of students	Reject input
Extremely large student count	Handle safely or reject
ID entered as 0	Reject and re-prompt
Negative ID	Reject and re-prompt
Non-numeric ID	Clear input and re-prompt
Duplicate student ID	Warn user and re-enter
Empty student name	Re-prompt
Name containing only spaces	Re-prompt
Marks < 0	Re-prompt
Marks > 100	Re-prompt
Non-numeric marks	Clear input and re-prompt
User presses Enter without input	Input rejected safely

2. Program Logic Edge Cases

Scenario	Expected Behavior
Display students when none exist	Display “No students available”
Find highest scorer with no students	Display warning
Display class summary with no students	Display safe message
Two students share highest average	Display first found
Average exactly equals passing threshold	Status = PASS
All students fail	Summary calculated correctly
All students have identical marks	Grades calculated consistently
Exit selected immediately	Program exits cleanly

3. File Handling Edge Cases

Scenario	Expected Behavior
File does not exist	Warning displayed, program continues
File exists but is empty	Display “No records available”
Partial or corrupt file	Skip invalid records
Invalid marks in file	Skip affected student
Invalid enum values	Skip record
Duplicate IDs in file	Warn and skip
File read interrupted	Program continues safely
Save with no students	Display warning
Load after adding students	Existing data replaced safely

4. Memory & Resource Edge Cases

Scenario	Expected Behavior
Adding students multiple times	No memory leaks
Loading file after adding students	Old memory released automatically
Exit without saving	Program exits cleanly
Large dynamic arrays	Smart pointers manage memory safely
Smart pointer reset	Old memory freed automatically

Lab File Structure:

File	Purpose
main.cpp	Menu and program flow
Student.h	Struct, enums, constants, function prototypes
Student.cpp	Function definitions, file handling, validation
CMakeLists.txt	Optional compilation instructions
.git	Version control

Git Commands for Lab Workflow:

```
git add .

git commit -m "Completed Lab 2: Advanced Student Management
System"
```

- Zip the entire project folder including .git
- Submit on Brightspace → Activities → Lab2

Sample (Correct) Output:

===== Student Management System =====

```
1. Add student records

2. Display all students

3. Find highest scoring student

4. Display class summary

5. Save records to file

6. Load records from file

7. Exit

Enter your choice: 1

Enter number of students: 2

Student 1:

ID: 101

Name: Alice

Marks (3 subjects): 85 90 78

Student 2:

ID: 102

Name: Bob

Marks (3 subjects): 65 70 75

===== Menu =====
```

```
2. Display all students

ID Name Avg Grade Status

---------------------------------------

101 Alice 84.33 A PASS

102 Bob 70.00 B PASS

Class Summary:

Students: 2

Class Average: 77.17

Passed: 2 / 2
```

CHECKLIST

General

- [ ] Lab 1 functionality preserved
- [ ] Code compiles without warnings
- [ ] No memory leaks
- [ ] .git folder included

Modern C++ Features

- [ ] Status unscoped enum
- [ ] Grade scoped enum
- [ ] constexpr used
- [ ] static variable used
- [ ] volatile variable used
- [ ] auto used appropriately
- [ ] static_cast used, no C-style casts

Memory Management

- [ ] std::unique_ptr<Student[]> used
- [ ] No raw delete in main()

Loops & Traversal

- [ ] Index-based loop used
- [ ] Range-based loop used
- [ ] Iterator-based loop used

Input & Output

- [ ] Menu input validated
- [ ] Student data validated
- [ ] Marks between 0–100
- [ ] Output formatted with <iomanip>

File Handling

- [ ] Records saved to file
- [ ] File load errors handled gracefully
- [ ] Corrupt records skipped

Menu Functionality

- [ ] Add students
- [ ] Display students
- [ ] Find highest scorer
- [ ] Class summary
- [ ] Save to file
- [ ] Load from file
- [ ] Exit cleanly